# Module 4: Finding, Downloading, and Applying Software on CURC Resources

# Session Overview

- The Module System (Lmod)
- Virtual Environments with Mamba and Conda
- Requesting Software Installations
- Advanced topics (if time permits)
  - Simplifying Source Installations with Spack
  - Containerization with Apptainer

# The Module System


Typically, HPC systems have preinstalled software available to all users. Some of the advantages of using this software are:

- Quick access to commonly used software
- Eliminates the need to self-install complex software
- Access to Software that is compatible with the system and often optimized for the hardware it runs on

A module on an HPC system provides easy access to this pre-installed software. We utilize **Lmod** to manage these modules and make them available to our users.

# The Module System


After logging into the HPC system, execute the following lines. These lines show you what modules are available on Login nodes and the Alpine compute nodes

```
$ module avail
$ acompile --help
$ acompile --time=2:00:00
$ module avail
```

As we can see, Login nodes **do not** have the full software stack

# The Module System


Live Demo: Loading and unloading software modules will set (and reset) important environment variables for you and will dynamically change the software environment on the cluster.

```
$ module purge
$ module load intel
$ module load hdf5
$ module display hdf5
$ env | grep HDF5
```

```
$ module purge
$ module load intel/2024.2.1
$ module avail
$ module load hdf5/1.14.5
$ module avail
```

# The Module System


Live Demo: Useful Lmod commands

```
module spider                          # list all available modules
module avail                           # list modules available to you
module load <package/version>          # load a module into your env
module purge                           # unload all modules
module list                            # list currently loaded modules
module display <package>               # display module info/help
module spider <package>                # view info for all versions
module spider <package/version>        # view info for specific version
```

# The Module System


Points to note about CURC-managed modules:

- CURC does not update system modules; we do fresh installs of new versions and change the default when that is appropriate
- Sometimes when a module is outdated or incompatible with the system, we will remove it from the software stack

**Take home: pay attention to what software modules you are loading, as this may be important for reproducibility!**

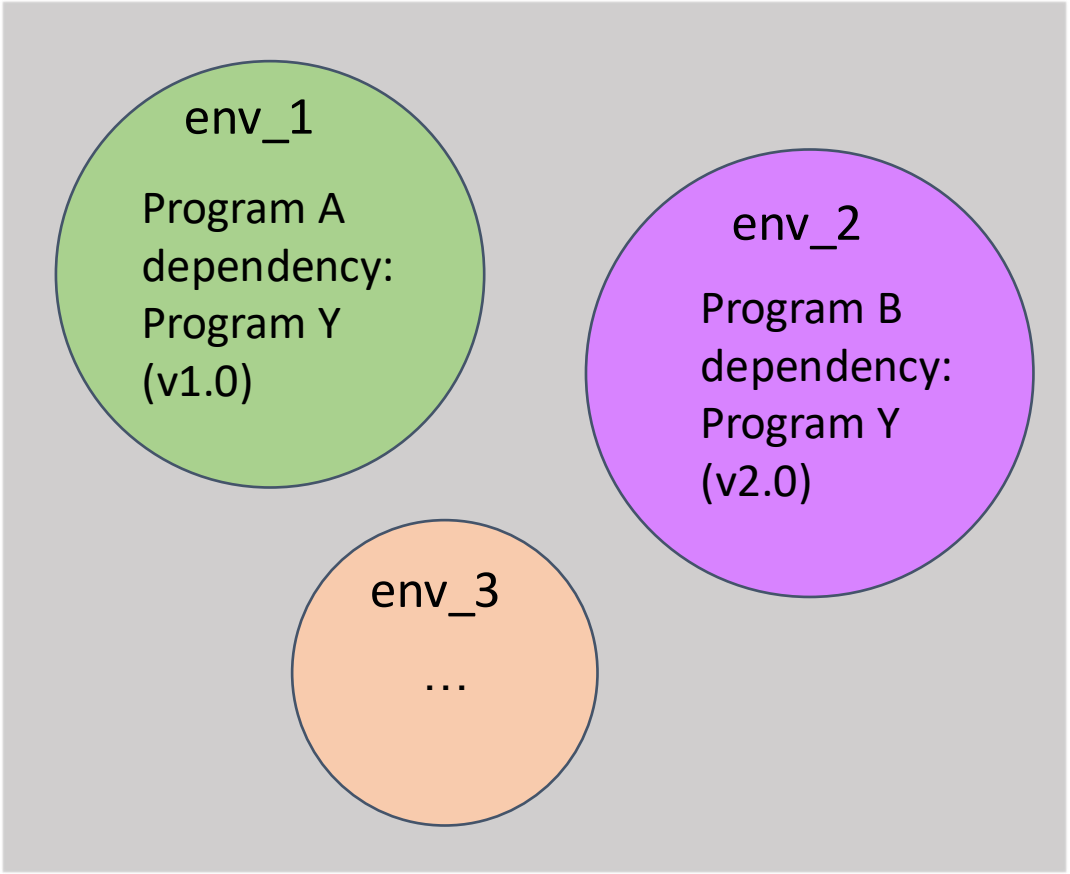
# Virtual Environments

Think of virtual environments as self-contained bubbles.

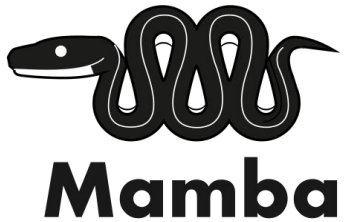
env_1 contains all the dependencies of ‘Program A’.

env_2 contains all the dependencies of ‘Program B’.

The environments do not interact.



# Virtual Environments



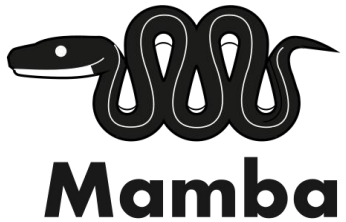


Mamba and Conda are package (software) managers that allow you to

- Install, run, and update packages (and their dependencies)
- Create, save, load, and switch between virtual environments
- Package and distribute software for any language

Mamba is a fast, robust, and more reliable cross-platform package manager that aims to be a drop-in replacement for Conda

# Virtual Environments





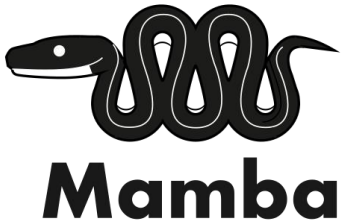
To access Mamba or Conda on our system, you should load the Miniforge module (once you are on a compute node):

```
$ module load miniforge
```

We highly recommend that you use our module as we ensure that installations are redirected to the appropriate places. For example, we ensure environments are installed in:

```
/projects/$USER/software/anaconda/envs
```

# Virtual Environments



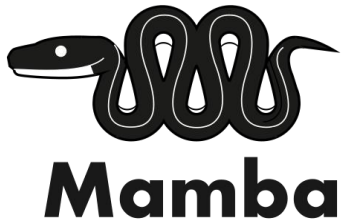


Environments are created and programs are installed in a few simple steps

```
$ module load miniforge
(base)$ mamba create -n my_first_env python==3.10
(base)$ mamba activate my_first_env
(my_first_env)$ python
```

Warning: Don’t install packages in your base environment!

# Virtual Environments





Packages are installed within **activated** environments

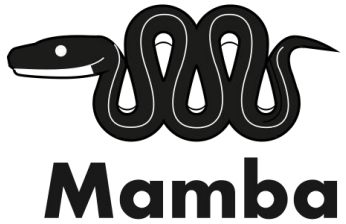
- using mamba install (preferred method, when available)
- using pip install (if you must)

```
$ mamba install pandas          #install latest pandas
$ mamba install pandas==2.3.3   #install specific version of pandas
```

```
$ pip install --no-cache-dir pandas       #install latest pandas
```

Warning: --no-cache-dir is crucial on CURC systems!

# Virtual Environments





Some useful commands (“mamba” can be replaced with “conda”)

```
mamba env list                          # list all environments
mamba list                              # list packages in active env
mamba env remove -n <envname>           # remove an environment
mamba config --show channels            # view configured channels
mamba deactivate                        # deactivate environment
mamba create --name <clonedenv> /       # clone an environment
      --clone <envtoclone>
```

# Want to go the extra mile?

Try our Hands-on exercise provided in EXERCISES.md. This is not required for the micro-credential.

**Objectives:**

1. Create a mamba environment and install samtools
2. Activate the environment and run samtools.

**Estimated time to complete**: 15 minutes

**Documentation:**

**https://curc.readthedocs.io/en/latest/software/python.html**

# Requesting Software Installations

- Is the software already installed on the cluster?

https://curc.readthedocs.io/en/latest/software/curc_provided_software.html

- Have you considered its utility and complexity?
  - Are you the only user of this software?
  - How complex or difficult is this software to install?
- Have you tried installing the package on your own?
- For help installing software, submit a support request form:

https://colorado.service-now.com/req_portal?id=ucb_sc_rc_form

# Advanced Topics

# Simplifying Installations with  Spack

How can we simplify source installations?

- **Package Managers** – Tools that automate installing, maintaining, and configuring software and any dependencies
- **Environments** – A collection of resources that are available in a self-contained 'bubble'

# Simplifying Installations with  Spack

Environments are created and programs are installed in a few simple steps

```
$ module purge
$ module load spack
$ spack env create my_first_env
$ spacktivate my_first_env
$ spack install --add samtools
```

Warning: Don’t install packages outside of an environment!

# Simplifying Installations with  Spack

Packages are installed within **activated** environments using Spack install

```
$ spack install --add samtools      #install default samtools
$ spack install --add samtools@1.9  #install specific version
```

# Simplifying Installations with  Spack

Spack installations can be very slow but will progress more quickly with the addition of cores.

- Spack builds all packages in parallel.
- The default parallelism is equal to the number of cores available to the process, up to 16.

# Simplifying Installations with  Spack

## Useful Spack commands

```
spack env list                      # list all your environments
spack remove <env>                  # remove an environment
spack uninstall <packagename>       # remove package
spack env status                    # check which env you're in
spack info <packagename>            # prints detailed package info
spack find                          # show installed packages
despacktivate                       # deactivate environment
spack spec <packagename>            # list packages plan
```

# Simplifying Installations with  Spack

- Useful Spack file paths

```
# root of the Spack install tree
/projects/$USER/software/spack


# location of package executables - these are symbolically linked to
      the installation tree subdirectory
/projects/$USER/spack/environments/<env>/.spack-env/view/bin


# location of Spack config file
/home/$USER/.spack/config.yaml
```

# Want to go the extra mile?

Try our Hands-on exercise provided in EXERCISES.md. This is not required for the micro-credential.

**Objectives:**

1. Create a Spack environment
2. Install fastqc in your Spack environment

**Estimated time to complete**: 20 minutes

**Documentation:**

**https://curc.readthedocs.io/en/latest/software/spack.html**

# Containerization with  APPTAINER

Containers are portable virtualizations of an operating system, software, libraries, data, and/or workflows

- Pros
  - Portability- they can run on any system equipped with its specified container manager
  - Reproducibility- they are instances of prebuilt isolated software; the software will always be the same every time
- Cons
  - Steeper learning curve than Mamba, Conda, and Spack
  - Can be difficult to troubleshoot issues
  - Building containers can be tricky for multi-node MPI applications

# Containerization with  APPTAINER

- CURC offers Apptainer (formerly Singularity) as container management software
  - Apptainer comes pre-installed on all Alpine nodes, so no need to load any specific software modules!
- Many common research applications have already been containerized and can be pulled from container repositories (such as Docker Hub).

# Containerization with APPTAINER

Useful Apptainer commands:

```
apptainer exec          #Execute a command to your container
apptainer run           #Run your image as an executable
apptainer build         #Build a container
apptainer pull          #pull an image from hub
apptainer inspect       #See labels/environment vars, run scripts
apptainer shell         #Access the command line of your container
```

# Containerization with APPTAINER

A container has its own file system and so needs help “seeing” files outside the container (on the host system). If not done in the .def file, this can be accomplished at runtime with bind mounting:

```
$ apptainer run -B /source/directory:/target/directory sample-image.sif
```

On CURC systems, a running container automatically bind mounts all major filesystems (/home, /projects, /scratch, and /pl/active). *If you are having an issue seeing a directory, you may need to bind mount it.*

# Want to go the extra mile?

Try our Hands-on exercise provided in EXERCISES.md. This is not required for the micro-credential.

**Objectives:**

1. Become familiar with basic Apptainer commands.
2. Pull an image from a pre-built container, then run the program from the container.

**Estimated time to complete**: 20 minutes

**Documentation:**

**https://curc.readthedocs.io/en/latest/software/containerization.html**

# Any Questions